ME 4504

MECHANICS OF MACHINES LAB

GROUP 4

Tepee simulation or 1 DOF mechanism

Prepared by

200012105 - Md Ibrahim Hossain Khan

200012123 - Rohan Shadman Arif

200012132 - Sadiq Mohammad Pritom

200012133 - Maysha Anan

200012146 - Naoshin Anzum Islam

200012153 - Zahin Tasnim Ridita

Date of Submission:

December 1st, 2023

Contents

Abstract 3
Introduction 3
Background 3
Kinematic Diagram 4
Mobility and Degrees of Freedom 4
Links and Angle Inputs 4
Calculation Related to the System 5
Vector loop equation 5
Velocity analysis 6
Acceleration analysis 6
Newton's method 6
MATLAB Codes 8
Function for Tepee 8
MATLAB Code for Tepee Simulation 8
Conditions 11
Initial Condition 11
Final Condition 11
Position vs Angular Displacement Graph 12
Velocity vs Angular Displacement Graph 12
Acceleration vs Angular Displacement Graph 13
Verification 13
Conclusion 13
References 14
Appendix A: Revision Notes 15

Abstract

A single-degree-of-freedom (1-DOF) tepee mechanism is a common mechanical linkage that converts rotary motion into reciprocating motion. This paper presents a MATLAB-based numerical simulation method for analyzing the kinematics of a 1-DOF tepee mechanism over the input range $\theta_2 = 80°$ to $0°$. The proposed method provides a valuable tool for understanding and analyzing the behavior of tepee mechanisms.

The MATLAB-based numerical simulation method utilizes vector loops and Newton's method to solve the position, velocity and acceleration equations of the tepee mechanism. Vector loops are employed to represent the kinematic relationships between the mechanism's links. Newton's method is used to iteratively solve for the unknown variables, such as the angular position and the linear position of the links. The proposed MATLAB-based numerical simulation method provides an efficient and accurate way to analyze the trajectory of a tepee mechanism: the computed slider position, velocity and acceleration agree with the closed-form solution of the mechanism (see Verification). The method is readily applicable to a wide range of tepee mechanism configurations. The simulation results offer valuable insights into the mechanism's motion and performance.

Introduction

A kinematic study of a one-DOF tepee mechanism is carried out in this report. It includes the position, velocity and acceleration analysis over the input range of the tepee mechanism, $\theta_2 = 80°$ to $0°$. A dynamic (force) analysis is not part of this report.

Background

Now, what are the uses of a tepee mechanism? The tepee mechanism is a slider-crank mechanism whose crank r_2 and coupler r_3 have equal length, so its slider moves in simple harmonic motion. Slider-crank-type mechanisms are widely applied in industry, for purposes such as internal combustion engines, pumps, robotics, toy cars etc.

This mechanism is sometimes also called 'Scotch Yoke'. Its slider has the same simple harmonic motion as a Scotch yoke, but it is a linkage: a Scotch yoke drives its slider with a pin that slides in a slot.

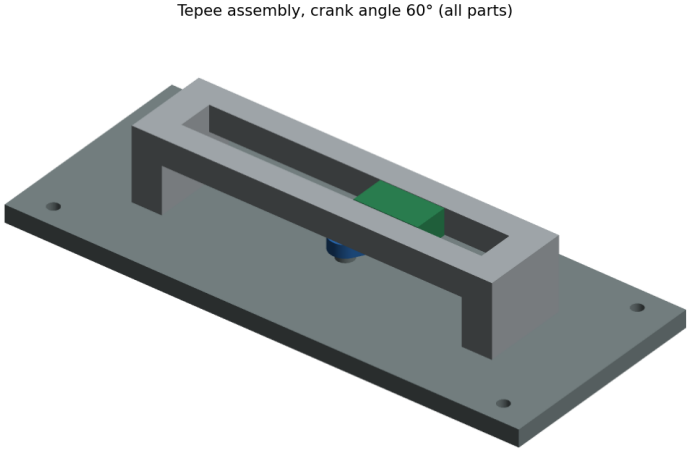


Figure 1: CAD model of the tepee mechanism at $\theta_2 = 60°$ (scaled model, $r_2 = r_3 = 25$ mm).

Kinematic Diagram

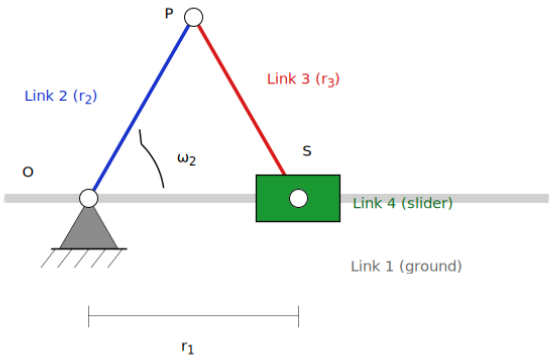


Figure 2: Kinematic diagram of the tepee mechanism: link 1 = ground, link 2 = crank r_2, link 3 = coupler r_3, link 4 = slider.

In order to begin our analysis, we will need to quantify the input values for velocity and acceleration. These values can be found listed in Table 1.

Mobility and Degrees of Freedom

The Degrees-Of-Freedom (DOF) of a device describes the number of ways that it can move. From our understanding of the tepee simulation mechanism, a DOF of 1 is desired, so that a single input angle fixes the position of every link. We can ensure that this is in fact the case by using Kutzbach's Mobility Equation (EQ. 1) for Planar Jointed Devices to confirm the DOF.

$$M = 3(N-1) - 2J_1 - J_2 \quad \text{(EQ. 1)}$$

In this Mobility Equation we have several variables to define. Here,

N = The number of links, including the ground link.
J_1 = The number of one-DOF joints (e.g. revolute or prismatic joints).
J_2 = The number of two-DOF joints (e.g. cam or gear joints).

The tepee mechanism that is being analyzed has a total of $N = 4$ links (ground, crank, coupler and slider). The mechanism is composed of 4 one-DOF joints J_1: three revolute joints (at O, P and S) and one prismatic (sliding) joint between the slider and the ground. It can be proven that the mobility will be 1, just as we were expecting.

$$M = 3(4-1) - 2 \cdot 4 - 0 = 1$$

Links and Angle Inputs

When we look at a snapshot of this mechanism and analyze the kinematics, we want to be sure that the inputs we choose are meaningful. The ground link lies along the x-axis ($\theta_1 = 0$), and the position input is represented here by θ_2 (theta_start) to θ_2 (theta_end). The link inputs are r_2 and r_3. We have to calculate the slider position r_1 and the angle θ_3 (theta3). Suppose links are in mm (millimeters) and theta is in radian units.

Table 1: Input information

Input information	Symbol	Value
Link 1 (slider position)	r_1	To be calculated
Link 2	r_2	10 mm
Link 3	r_3	10 mm ($= r_2$)
Input angle (start)	θ_2	80 degree
Input angle (end)	θ_2	0 degree
Input angular velocity	$\dot{\theta}_2$	1 rad/s (constant)
Input angular acceleration	$\ddot{\theta}_2$	0 rad/s²
Number of frames	n	30
Newton tolerance	tol	10^{-10}

Calculation Related to the System

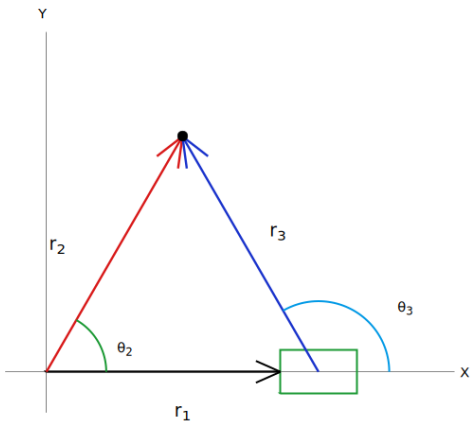


Figure 3: Vector loop of the tepee mechanism.

Vector loop equation

With $\vec{r}_1$ directed from O to the slider, $\vec{r}_3$ from the slider to P and $\vec{r}_2$ from O to P (see the figure),

$$\vec{r}_1 + \vec{r}_3 - \vec{r}_2 = \vec{0}.$$

X-axis:

$$r_1\cos\theta_1 + r_3\cos\theta_3 - r_2\cos\theta_2 = 0 \;\Rightarrow\; r_1 + r_3\cos\theta_3 - r_2\cos\theta_2 = 0 \tag{1}$$

Y-axis:

$$r_1\sin\theta_1 + r_3\sin\theta_3 - r_2\sin\theta_2 = 0 \;\Rightarrow\; r_3\sin\theta_3 - r_2\sin\theta_2 = 0 \tag{2}$$

(with $\theta_1 = 0$, so $\cos\theta_1 = 1$ and $\sin\theta_1 = 0$).

Knowns: r_2, r_3, $\theta_1 = 0$, and $\theta_3 = 180° - \theta_2$ (this holds because $r_2 = r_3$, which makes the triangle formed by the links isosceles).
Input: θ_2. Unknowns: r_1, θ_3.
Links: 4. Joints: 4 (3 rotational, 1 sliding).

From Equation (1),

$$r_1 = r_2\cos\theta_2 - r_3\cos(180^\circ - \theta_2) = 2\,r_2\cos\theta_2.$$

Velocity analysis

Differentiating (1) and (2) with respect to time (r_2 and r_3 are constant):

$$\begin{aligned} \dot{X}: &\quad \dot{r}_1 - r_3\sin\theta_3\,\dot{\theta}_3 + r_2\sin\theta_2\,\dot{\theta}_2 = 0 \\ \dot{Y}: &\quad r_3\cos\theta_3\,\dot{\theta}_3 - r_2\cos\theta_2\,\dot{\theta}_2 = 0 \end{aligned}$$

In matrix form,

$$\begin{bmatrix} 1 & -r_3\sin\theta_3 \\ 0 & r_3\cos\theta_3 \end{bmatrix}\begin{bmatrix} \dot{r}_1 \\ \dot{\theta}_3 \end{bmatrix} = \begin{bmatrix} -r_2\sin\theta_2\,\dot{\theta}_2 \\ r_2\cos\theta_2\,\dot{\theta}_2 \end{bmatrix}.$$

Solving,

$$\dot{\theta}_3 = \frac{r_2\cos\theta_2}{r_3\cos\theta_3}\,\dot{\theta}_2, \qquad \dot{r}_1 = r_3\sin\theta_3\,\dot{\theta}_3 - r_2\sin\theta_2\,\dot{\theta}_2 = \frac{r_2\sin(\theta_3 - \theta_2)}{\cos\theta_3}\,\dot{\theta}_2.$$

For the tepee mechanism ($r_2 = r_3$, $\theta_3 = 180^\circ - \theta_2$): $\dot{\theta}_3 = -\dot{\theta}_2$ and $\dot{r}_1 = -2\,r_2\sin\theta_2\,\dot{\theta}_2$.

Acceleration analysis

Differentiating once more:

$$\begin{aligned} \ddot{X}: &\quad \ddot{r}_1 - r_3\sin\theta_3\,\ddot{\theta}_3 - r_3\cos\theta_3\,\dot{\theta}_3^2 + r_2\sin\theta_2\,\ddot{\theta}_2 + r_2\cos\theta_2\,\dot{\theta}_2^2 = 0 \\ \ddot{Y}: &\quad r_3\cos\theta_3\,\ddot{\theta}_3 - r_3\sin\theta_3\,\dot{\theta}_3^2 - r_2\cos\theta_2\,\ddot{\theta}_2 + r_2\sin\theta_2\,\dot{\theta}_2^2 = 0 \end{aligned}$$

Now, we will form a Jacobian matrix with these two equations:

$$\begin{bmatrix} 1 & -r_3\sin\theta_3 \\ 0 & r_3\cos\theta_3 \end{bmatrix}\begin{bmatrix} \ddot{r}_1 \\ \ddot{\theta}_3 \end{bmatrix} = \begin{bmatrix} ba_1 \\ ba_2 \end{bmatrix},$$

$$ba_1 = -r_2\cos\theta_2\,\dot{\theta}_2^2 - r_2\sin\theta_2\,\ddot{\theta}_2 + r_3\cos\theta_3\,\dot{\theta}_3^2,$$

$$ba_2 = -r_2\sin\theta_2\,\dot{\theta}_2^2 + r_2\cos\theta_2\,\ddot{\theta}_2 + r_3\sin\theta_3\,\dot{\theta}_3^2.$$

Here, $\ddot{\theta}_3 = ba_2/(r_3\cos\theta_3)$ and $\ddot{r}_1 = ba_1 + r_3\sin\theta_3\,\ddot{\theta}_3 = ba_1 + ba_2\tan\theta_3$. For the tepee mechanism with $\ddot{\theta}_2 = 0$: $\ddot{\theta}_3 = 0$ and $\ddot{r}_1 = -2\,r_2\cos\theta_2\,\dot{\theta}_2^2$.

Newton's method

Newton's method is applied to the position equations (1) and (2), which are nonlinear in θ_3. With the vector of unknowns $X = [\,r_1 \;\; \theta_3\,]^T$,

$$F(X) = \begin{bmatrix} r_1 + r_3\cos\theta_3 - r_2\cos\theta_2 \\ r_3\sin\theta_3 - r_2\sin\theta_2 \end{bmatrix}, \qquad J = \frac{\partial F}{\partial X} = \begin{bmatrix} 1 & -r_3\sin\theta_3 \\ 0 & r_3\cos\theta_3 \end{bmatrix}.$$

We plug Newton's method into our MATLAB code and repeat the iteration for each input angle until $\|\,F\,\|$ is below the tolerance. The initial guess is $r_1 = 0$, $\theta_3 = \pi/2 + 0.1$ for the first frame, and the solution of the previous frame afterwards. The velocity and acceleration equations are linear in their unknowns and are solved directly with the same Jacobian.

$$X_{n+1} = X_n - J^{-1}F(X_n)$$

- X_{n+1} is the updated vector of variables.
- X_n is the current vector of variables.
- J^{-1} is the inverse of the Jacobian matrix.
- $F(X_n)$ is the vector of function values at the current iteration.

J is singular when $\cos\theta_3 = 0$ ($\theta_2 = 90°$ for the tepee mechanism); the simulated input range stays clear of this position.

MATLAB Codes

Function for Tepee

```
function [r1, t3] = solveTeepee(r2, r3, t2)
% Solve unknowns for teepee mechanism
% INPUTS: knowns and mechanism input: r2, r3, t1, t2
% OUTPUTS: unknowns: r1, t3
% solve for theta3
t3 = pi - t2;
% solve for r1 second
r1 = r2*cos(t2) - r3*cos(t3);
end
```

MATLAB Code for Tepee Simulation

Lines changed in this edition carry the comment “REVISED” (see Appendix A).

```
%Tepee Simulation Code
%clear stuff
close all
clear
clc
%define knowns
r2 = 10; % tepee length (mm)   % REVISED: unit (was ft)
r3 = r2;
t1 =0;
tol = 1e-10; % REVISED: was 0.001
%define inputs
t2start =80*(pi/180); %theta(rad)
t2end = 0*(pi/180);
n = 30; %number of frames to count
t2 = linspace(t2start, t2end, n)';
t2d = 1; %velocity
t2dd = 0; %acceleration
%preallocate arrays
r1 = zeros(n,1);
t3 = zeros(n,1);
r1d = zeros(n,1);
t3d = zeros(n,1);
r1dd = zeros(n,1);
t3dd = zeros(n,1);
% initial guess
r1g = 0;
t3g = pi/2 +0.1;
guess = [r1g; t3g];
%calculate unknowns
for ct = 1:n

 %calc vector loop position scalars
 x = r1g- r2*cos(t2(ct)) + r3*cos(t3g);
 y = - r2*sin(t2(ct)) + r3*sin(t3g);
 f = [x; y];

 %calc error
 err = norm(f);

 %guess and check
 it = 0; % REVISED: iteration counter
 while err > tol && it < 50 % REVISED: iteration cap

 %jacobian

 J = [1, -r3*sin(t3g);...
 0, r3*cos(t3g)];

 %new guess
 guess = guess - J\f;
 r1g = guess(1);
 t3g = guess(2);

 % re-clac position scalars
 x = r1g- r2*cos(t2(ct)) + r3*cos(t3g);
 y = - r2*sin(t2(ct)) + r3*sin(t3g);
 f = [x; y];

%re-clac error
```

```
 err = norm(f);
 it = it + 1; % REVISED
 end

 %save answers
 r1(ct) = r1g;
 t3(ct) = t3g;
%calculate velocity and acceleration
 %jacobian
 J = [1, -r3*sin(t3(ct));...
 0, r3*cos(t3(ct))];

 %velocity
 bv = [-r2*sin(t2(ct))*t2d; r2*cos(t2(ct))*t2d];
 thetadot = J\bv;
 r1d(ct) = thetadot(1);
 t3d(ct) = thetadot(2);

 %acceleration
 ba(1) = -r2*cos(t2(ct))*t2d^2-r2*sin(t2(ct))*t2dd+r3*cos(t3(ct))*t3d(ct)^2;
 ba(2) = -r2*sin(t2(ct))*t2d^2+r2*cos(t2(ct))*t2dd+r3*sin(t3(ct))*t3d(ct)^2; % REVISED: sign of
t2dd term
 thetaddot = J\ba'; % REVISED: matrix solve (was inv(J).*ba, an element-wise product)
 r1dd(ct) = thetaddot(1);
 t3dd(ct) = thetaddot(2);

end

%verification against the closed-form solution (valid because r2 = r3)   % REVISED: added
[r1c, t3c] = solveTeepee(r2, r3, t2);
fprintf('max |r1 - closed form|     = %.2e\n', max(abs(r1 - r1c)));
fprintf('max |theta3 - closed form| = %.2e\n', max(abs(t3 - t3c)));
fprintf('max |r1d - closed form|    = %.2e\n', max(abs(r1d + 2*r2*sin(t2)*t2d)));
fprintf('max |r1dd - closed form|   = %.2e\n', max(abs(r1dd + 2*r2*cos(t2)*t2d^2)));
for ct = 1:n
%calculate joint location
p0x = 0;
p0y = 0;
p23x = r2*cos(t2(ct));
p23y = r2*sin(t2(ct));
p13x = r1(ct);
p13y = 0;
%calculation lines
leg2x = [p0x p23x];
leg2y = [p0y p23y];
leg3x = [p13x p23x];
leg3y = [p13y p23y];
floorx = [p0x p13x];
floory = [p0y p13y];
%plot mechanism
figure(1)
clf
hold on
plot(leg2x, leg2y, 'b')
plot(leg3x, leg3y, 'r')
plot(floorx, floory, 'g')
xlim([0 20])
ylim([0 15])
pause(0.1)
end
%%Static Plots
% Position
Pos = [r1, t3];
figure(2)
hold on
plot(t2,Pos, 'LineWidth', 2)
xlabel('\theta_2 (rad)')
ylabel('positions')
name = legend('${r}_{1}$ (mm)','${\theta}_{3}$ (rad)'); % REVISED: unit
set(name,'Interpreter','latex');
title('Joint Positions')
pause(0.5)
%Velocity
Vel = [r1d , t3d];
figure(3)
hold on
plot(t2,Vel,'LineWidth',2)
xlabel('\theta_2 (rad)')
```

```
ylabel('velocities')
name = legend('$\dot{r}_{1}$ (mm/s)','$\dot{\theta}_{3}$ (rad/s)'); % REVISED: unit, stray $
removed
set(name,'Interpreter','latex');
title('Joint Velocities')
pause(0.5)
%acceleration
Accel = [r1dd, t3dd];
figure(4)
hold on
plot(t2,Accel, 'LineWidth', 2)
xlabel('\theta_2 (rad)')
ylabel('acceleration')
name = legend('$\ddot{r}_{1}$ (mm/s^2)','$\ddot{\theta}_{3}$ (rad/s^2)'); % REVISED: unit, stray
$ removed
set(name,'Interpreter','latex');
title('Joint Accelerations')
pause(0.5)
```

Conditions

Initial Condition

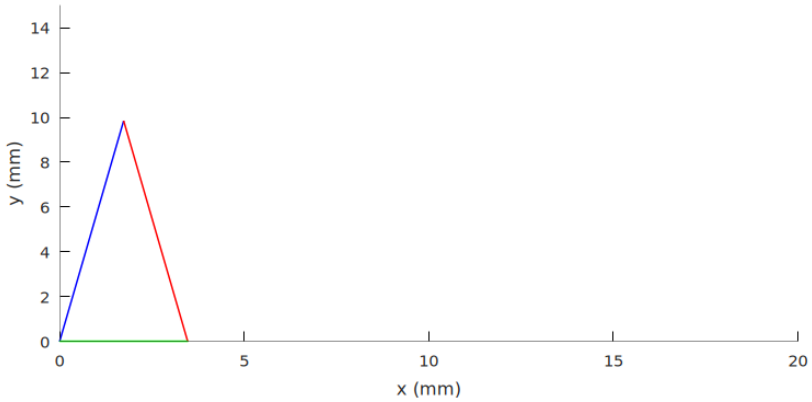


Figure 4: Angle at 80 degree.

Final Condition

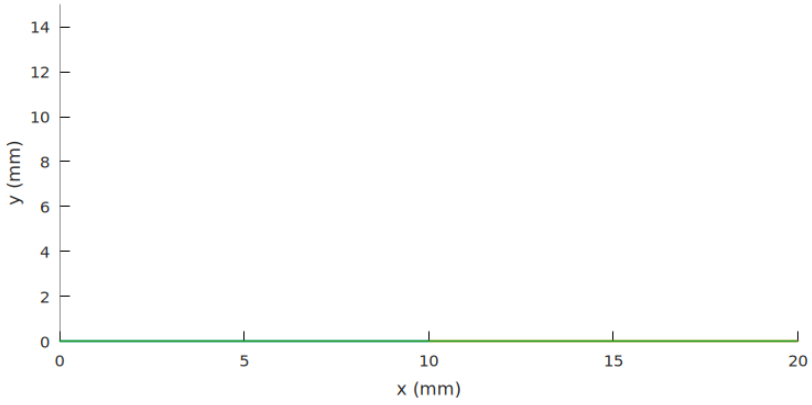


Figure 5: Angle at 0 degree.

Position vs Angular Displacement Graph

The plot is designed to illustrate how the joint positions r_1 and θ_3 vary as a function of the input variable θ_2 over the specified range. This visualization allows users to observe the behavior of the mechanism in terms of joint positions as the input variable changes. It provides insights into how the mechanism moves and reaches different configurations. From the graph, it can be seen that, the position of r_1 (BLUE) is decreasing as θ_2 increases, from 20 mm to 3.47 mm, and follows $r_1 = 2r_2\cos\theta_2$ (a cosine curve, close to parabolic for small θ_2), whereas θ_3 (RED) is also decreasing, as a straight line, $\theta_3 = \pi - \theta_2$.

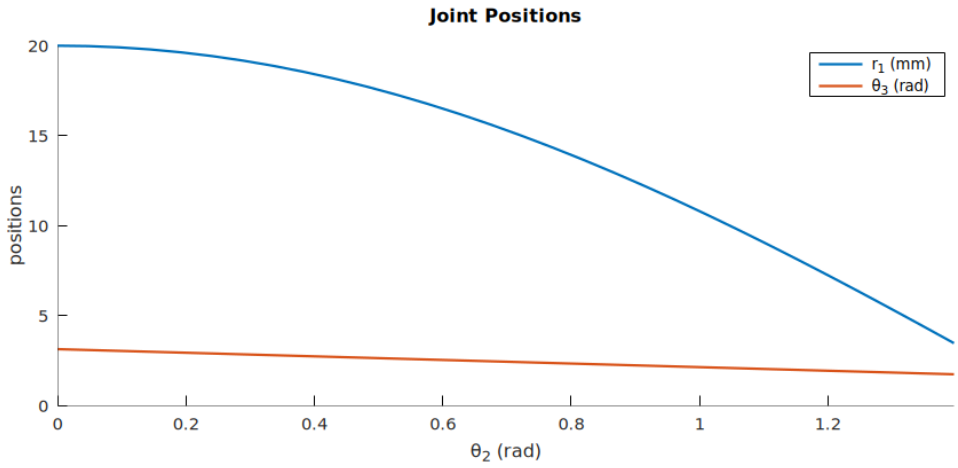


Figure 6: Position vs angular position graph.

Velocity vs Angular Displacement Graph

The plot is designed to illustrate how the joint velocities $\dot{r}_1$ and $\dot{\theta}_3$ change as a function of the input variable θ_2 over the specified range. This visualization allows users to observe the behavior of the mechanism in terms of joint velocities as the input variable changes. It provides insights into the speed and direction of motion at different configurations. From the graph, it can be seen that, the velocity of r_1 (BLUE) is decreasing, from 0 to −19.7 mm/s, and follows $\dot{r}_1 = -2r_2\dot{\theta}_2\sin\theta_2$ (a sine curve), whereas the angular velocity of θ_3 (RED) is constant, −1 rad/s ($= -\dot{\theta}_2$), and the line is straight.

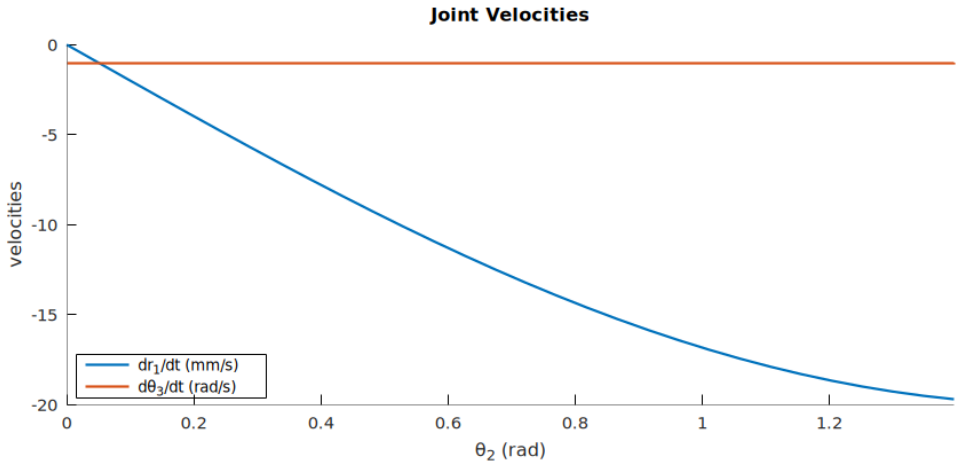


Figure 7: Velocity vs angular position graph.

Acceleration vs Angular Displacement Graph

The plot is designed to illustrate how the joint accelerations $\ddot{r}_1$ and $\ddot{\theta}_3$ change as a function of the input variable θ_2 over the specified range. This visualization allows users to observe the behavior of the mechanism in terms of joint accelerations as the input variable changes. It provides insights into how quickly the system is speeding up or slowing down in different configurations. From the graph, it can be seen that, the acceleration of r_1 (BLUE) is increasing, from −20 mm/s² to −3.47 mm/s², and follows $\ddot{r}_1 = -2r_2\dot{\theta}_2^2\cos\theta_2$ (a cosine curve), whereas the angular acceleration $\ddot{\theta}_3$ (RED) is zero, so there will be no change with respect to the changing angle θ_2.

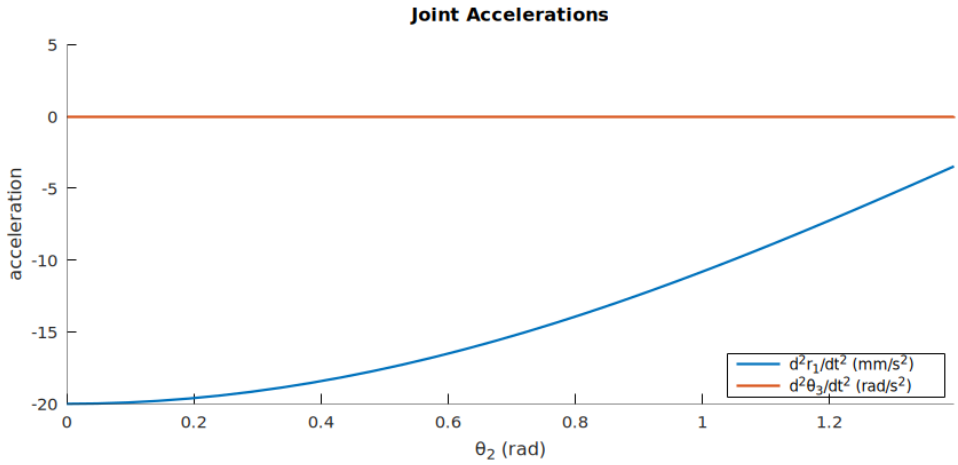


Figure 8: Acceleration vs angular position graph.

Verification

The Newton results were checked against the closed-form solution of the tepee mechanism, $r_1 = 2r_2\cos\theta_2$, $\theta_3 = \pi - \theta_2$, $\dot{r}_1 = -2r_2\dot{\theta}_2\sin\theta_2$ and $\ddot{r}_1 = -2r_2\dot{\theta}_2^2\cos\theta_2$ (the first two use the function `solveTeepee`; the check is part of the revised script). The table gives the largest difference over the 30 frames.

Table 2: Largest difference between the simulation and the closed-form solution

Quantity	Submitted script (tol 10^{-3})	Revised script (tol 10^{-10})
r_1 (mm)	9×10^{-4}	1×10^{-10}
θ_3 (rad)	9×10^{-5}	9×10^{-12}
$\dot{r}_1$ (mm/s)	4×10^{-3}	2×10^{-10}
$\ddot{r}_1$ (mm/s²)	9×10^{-4}	5×10^{-10}

The tolerance was tightened because the corrected acceleration equation divides by $\cos\theta_3$, which magnifies position error when θ_2 is large. End values for the simulated range: at $\theta_2 = 80°$, $r_1 = 3.473$ mm, $\dot{r}_1 = -19.696$ mm/s, $\ddot{r}_1 = -3.473$ mm/s²; at $\theta_2 = 0°$, $r_1 = 20$ mm, $\dot{r}_1 = 0$, $\ddot{r}_1 = -20$ mm/s².

Conclusion

The MATLAB code presented herein conducted a position, velocity, and acceleration analysis of a tepee mechanism using Newton's method and vector loop equations. By defining the tepee lengths, initial and final angles, and the number of frames, the code facilitated a kinematic simulation of the mechanism's motion. The iterative nature of Newton's method and the incorporation of vector loop equations enabled precise calculations of joint positions, velocities, and accelerations for each frame. The resulting joint

trajectories, along with their corresponding velocity and acceleration profiles, were systematically analyzed and visualized, and they agree with the closed-form solution of the mechanism. The iterative convergence checks played a pivotal role in ensuring the accuracy and stability of the simulations. The animated plots provided insight into the mechanical behavior of the tepee system, showcasing its motion across a specified range of input angles.

To adapt the code for a 2 or 3-DOF mechanism, enhancements could include implementing a more robust initialization strategy to improve the convergence of Newton's method. Additionally, extending the code to handle multiple links and joints, along with optimizing for vectorized operations, would enhance its versatility and efficiency in simulating complex robotic systems.

References

1. YouTube video (link as given in the original report): https://www.youtube.com/watch?v=L-r15ES3vFI&list=PLTMo3nrl2KWbeCx8-OGfrMHcFOUSjyk4D&index=8
2. *ME 3011 Final Report (Revised)*, hosted at jonathanmbowman.com (link as given in the original report): https://www.jonathanmbowman.com/pdfs/ME%203011%20Final%20Report%20Revised.pdf

Appendix A: Revision Notes

This edition corrects the report submitted on 1 December 2023. The method, the inputs, the results, the structure and the order of the sections are unchanged, and the original wording is kept wherever it was correct. Corrections were prepared in October 2026 by Zahin Tasnim Ridita with AI assistance (Claude, Anthropic). The code was run in GNU Octave 8.4.

No.	Where	Submitted version	Revised version and reason
1	Abstract, Introduction, Conclusion	"2DOF" / "two DOF" tepee mechanism	1-DOF. The report's own mobility calculation gives $M = 1$ and its title says 1 DOF.
2	Abstract, Introduction	"entire cycle of movement", "full range of motion"	The simulated input range, $\theta_2 = 80°$ to $0°$.
3	Abstract, Introduction, Conclusion	"kinematic and dynamic study", "equations of motion", "dynamic simulation"	Kinematic study; position, velocity and acceleration equations. No forces or inertia are computed.
4	Background	Industrial uses stated for the "tepee mechanism"; "sometimes also called Scotch Yoke"	Uses attributed to slider-crank-type mechanisms; the Scotch-yoke remark qualified (same harmonic slider motion, different mechanism).
5	Background, Kinematic Diagram	Two images with no stated source: a CAD picture of a different mechanism, and a generic crank-slider diagram in other notation ($r, L, \varphi, \beta, s_p$)	Replaced by original figures: a CAD render of the tepee and a kinematic diagram in the report's notation.
6	Mobility	"tepee simulation or the three bar mechanism"; "4 revolute joints"; a mechanism "has one and only one" DOF by definition	Four links; three revolute joints and one prismatic joint ($J_1 = 4$, as the report's joint count states later). The "by definition" remark removed.
7	Links and Angle Inputs	r_1, r_2 given as inputs, r_3 to be calculated	r_2, r_3 known; r_1, θ_3 unknown, as in Table 1 and the code. Ground-link angle $\theta_1 = 0$ stated.
8	Table 1	Start and end angles labelled θ_1 and θ_2; velocity and acceleration inputs missing although the text points to Table 1; "10 units"	Both angles labelled θ_2; $\dot{\theta}_2$, $\ddot{\theta}_2$, frame count and tolerance added; units in mm, as the text states (the code comment said ft and the plot legends m).
9	Vector loop	$\vec{r}_1 + \vec{r}_2 - \vec{r}_3 = \vec{0}$ and component equations whose signs disagree with the figure, the next line and the code	$\vec{r}_1 + \vec{r}_3 - \vec{r}_2 = \vec{0}$ and equations (1), (2) matching figure and code; list of unknowns corrected; note that $\theta_3 = 180° - \theta_2$ holds because $r_2 = r_3$.
10	Velocity analysis	Derivative equations containing $\dot{r}_2, \dot{r}_3$ (zero for rigid links) and an incomplete $\dot{Y}$ equation; $\dot{r}_1 = r_2\dot{\theta}_2\sin(\theta_3 - \theta_2)$	Rewritten from the derivatives; the factor $1/\cos\theta_3$ added to $\dot{r}_1$; tepee simplification added.
11	Acceleration analysis	Equations and right-hand side garbled; sign of the $\ddot{\theta}_2$ term in the $\ddot{Y}$ equation	Rewritten from the derivatives; sign corrected; tepee simplification added.
12	Newton's method	"plug Newton's method into our MATLAB code"	F, J, initial guess and tolerance stated; Newton used for position only (velocity and acceleration are linear); singular J noted.
13	Code, acceleration	`thetaddot = inv(J).*ba` (element-wise product)	`thetaddot = J\ba'` (matrix solve). Same output for $r_2 = r_3, \ddot{\theta}_2 = 0$, which is why the plots were right, but not for $r_2 \neq r_3$. It now implements $\ddot{r}_1 = ba_1 + ba_2\tan\theta_3$ from the report.
14	Code, `ba(2)`	$-r_2\cos\theta_2\,\ddot{\theta}_2$	$+r_2\cos\theta_2\,\ddot{\theta}_2$ (no effect while $\ddot{\theta}_2 = 0$).

No.	Where	Submitted version	Revised version and reason
15	Code, tolerance	`tol = 0.001`; no iteration limit	`tol = 1e-10`; at most 50 iterations. The corrected acceleration equation divides by $\cos\theta_3$, which magnifies position error; the loop could otherwise never end near $\theta_2 = 90°$.
16	Code, units and legends	Length comment in ft; legends in m and m/s; a stray `$` in two legends (shown as raw text in the figures)	mm, mm/s, mm/s²; stray `$` removed.
17	Code, verification	None (`solveTeepee` was defined but never called)	Verification lines added, using `solveTeepee` and the closed-form velocity and acceleration.
18	Figures	MATLAB screenshots with raw LaTeX legend text	Regenerated from the revised script with proper legends and axis labels; same data.
19	Graph descriptions	“parabolic”, “slope”; velocity “almost the same as the position graph”	Described by their actual forms (cosine, straight line, sine); end values given.
20	Verification	–	New section.
21	Contents, references	Contents page numbers; URLs broken across lines	Contents regenerated; reference list formatted; links unchanged.
22	Cover	–	Unchanged.